

Target Detection

Target Detection

1. Target Detection

1.1. Model Structure

1.2. Code Analysis

1.3. Startup

1. Target Detection

The main problem solved in this section is how to use the `dnn` module in OpenCV to import a pre-trained target detection network. However, there are requirements on the OpenCV version.

Currently, there are three main methods for target detection using deep learning:

- Faster R-CNNs
- You Only Look Once (YOLO)
- Single Shot Detectors (SSDs)

Faster R-CNNs are the most commonly heard neural network based on deep learning. However, this method is technically difficult to understand (especially for deep learning beginners), difficult to implement, and also difficult to train.

Moreover, even with the "Faster" method of implementing R-CNNs (here R stands for Region Proposal), the algorithm is still relatively slow, about 7 FPS.

If we pursue speed, we can turn to YOLO, because it is very fast, reaching 40-90 FPS on a Titan X GPU, and the fastest version can reach 155 FPS. But the problem with YOLO is that its accuracy still needs improvement.

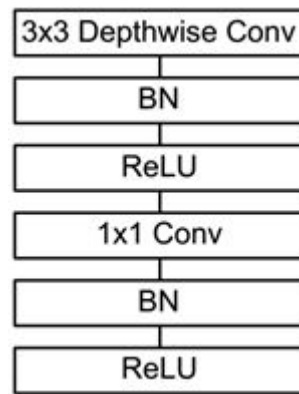
SSDs were originally developed by Google and can be said to be a balance between the above two. Compared with Faster R-CNNs, its algorithm is more direct. Compared with YOLO, it is more accurate.

1.1. Model Structure

The main work of MobileNet is to use depthwise separable convolutions to replace the past standard convolutions to solve the problems of computational efficiency and the number of parameters of the convolutional network. The MobileNets model is based on depthwise separable convolutions, which can decompose a standard convolution into a depthwise convolution and a pointwise convolution (1 x 1 convolution kernel). **The depthwise convolution applies each convolution kernel to each channel, while the 1 x 1 convolution is used to combine the outputs of the channel convolution.**

Batch Normalization (BN) is added to the basic components of MobileNet, that is, normalization processing is performed at each SGD (stochastic gradient descent), so that the mean of the result (the various dimensions of the output signal) is 0 and the variance is 1. Generally, when training a neural network encounters problems such as very slow convergence or gradient explosion that prevent training, you can try BN to solve them. In addition, in general use cases, BN can also be added to speed up training and improve model accuracy.

In addition, the model also uses the ReLU activation function, so the basic structure of the depthwise separable convolution is shown in the figure below:



The MobileNets network is composed of many depthwise separable convolutions as shown in the figure above. Its specific network structure is shown in the figure below:

Type / Stride	Filter Shape	Input Size
Conv / s2	$3 \times 3 \times 3 \times 32$	$224 \times 224 \times 3$
Conv dw / s1	$3 \times 3 \times 32$ dw	$112 \times 112 \times 32$
Conv / s1	$1 \times 1 \times 32 \times 64$	$112 \times 112 \times 32$
Conv dw / s2	$3 \times 3 \times 64$ dw	$112 \times 112 \times 64$
Conv / s1	$1 \times 1 \times 64 \times 128$	$56 \times 56 \times 64$
Conv dw / s1	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 128$	$56 \times 56 \times 128$
Conv dw / s2	$3 \times 3 \times 128$ dw	$56 \times 56 \times 128$
Conv / s1	$1 \times 1 \times 128 \times 256$	$28 \times 28 \times 128$
Conv dw / s1	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 256$	$28 \times 28 \times 256$
Conv dw / s2	$3 \times 3 \times 256$ dw	$28 \times 28 \times 256$
Conv / s1	$1 \times 1 \times 256 \times 512$	$14 \times 14 \times 256$
5×	Conv dw / s1	$3 \times 3 \times 512$ dw
	Conv / s1	$1 \times 1 \times 512 \times 512$
	Conv dw / s2	$3 \times 3 \times 512$ dw
	Conv / s1	$1 \times 1 \times 512 \times 1024$
	Conv dw / s1	$3 \times 3 \times 1024$ dw
	Conv / s1	$1 \times 1 \times 1024 \times 1024$
	Avg Pool / s1	Pool 7×7
	FC / s1	1024×1000
	Softmax / s1	Classifier

1.2. Code Analysis

The list of recognizable objects

```
[person, bicycle, car, motorcycle, airplane, bus, train,
truck, boat, traffic light, fire hydrant, street sign,
stop sign, parking meter, bench, bird, cat, dog, horse,
sheep, cow, elephant, bear, zebra, giraffe, hat, backpack,
umbrella, shoe, eye glasses, handbag, tie, suitcase,
frisbee, skis, snowboard, sports ball, kite, baseball bat,
baseball glove, skateboard, surfboard, tennis racket,
```

```
bottle, plate, wine glass, cup, fork, knife, spoon, bowl,  
banana, apple, sandwich, orange, broccoli, carrot, hot dog,  
pizza, donut, cake, chair, couch, potted plant, bed, mirror,  
dining table, window, desk, toilet, door, tv, laptop, mouse,  
remote, keyboard, cell phone, microwave, oven, toaster,  
sink, refrigerator, blender, book, clock, vase, scissors,  
teddy bear, hair drier, toothbrush]
```

Load the categories [object_detection_coco.txt], import the model [frozen_inference_graph.pb], and specify the deep learning framework [TensorFlow]

```
# Load COCO class names  
with open('object_detection_coco.txt', 'r') as f: class_names =  
f.read().split('\n')  
# Display different colors for different targets  
COLORS = np.random.uniform(0, 255, size=(len(class_names), 3))  
# Load the DNN image model  
model = cv.dnn.readNet(model='frozen_inference_graph.pb',  
config='ssd_mobilenet_v2_coco.txt', framework='TensorFlow')
```

Import the image, extract the height and width, compute a 300x300 pixel blob, and pass this blob to the neural network

```
def Target_Detection(image):  
    image_height, image_width, _ = image.shape  
    # Create a blob from the image  
    blob = cv.dnn.blobFromImage(image=image, size=(300, 300), mean=(104, 117,  
123), swapRB=True)  
    model.setInput(blob)  
    output = model.forward()  
    # Loop over each detection  
    for detection in output[0, 0, :, :]:  
        # Extract the confidence of the detection  
        confidence = detection[2]  
        # Draw a bounding box only when the detection confidence is higher than a  
        certain threshold, otherwise skip  
        if confidence > .4:  
            # Get the class ID  
            class_id = detection[1]  
            # Map the class ID to the class  
            class_name = class_names[int(class_id) - 1]  
            color = COLORS[int(class_id)]  
            # Get the bounding box coordinates  
            box_x = detection[3] * image_width  
            box_y = detection[4] * image_height  
            # Get the bounding box width and height  
            box_width = detection[5] * image_width  
            box_height = detection[6] * image_height  
            # Draw a rectangle around each detected object  
            cv.rectangle(image, (int(box_x), int(box_y)), (int(box_width),  
int(box_height)), color, thickness=2)  
            # Write the class name text on the detected object  
            cv.putText(image, class_name, (int(box_x), int(box_y) - 5)),  
cv.FONT_HERSHEY_SIMPLEX, 1, color, 2)  
    return image
```

1.3. Startup

```
cd /home/pi/yahboomcar_ws/src/yahboomcar_visual/detection
python target_detection_CSI.py (CSI camera)
python target_detection_USB.py (USB camera)
```

After clicking the image frame, press the [f] key on the keyboard to switch to human pose estimation.

```
if action == ord('f') or action == ord('F'):state = not state # function switch
```

